

UNIVERSIDAD TECNOLÓGICA DEL PERÚ

Facultad de Ingeniería de Sistemas e Informática

Mundial Trivia Challenge: Sistema interactivo de preguntas y análisis de rendimiento con NumPy

Informe técnico del proyecto

Proyecto final

Integrantes:	Mamani Aguilar, Luis Enrique	U23259985
	Cortez Benites, Eduardo Franco	U1421099
	Gallardo Villa, Paul Williams	U1614474
Asignatura:	Programación de Computadoras (100000I42M)	
Periodo:	2026 – Ciclo 1 (marzo)	

Lima, Perú

2026

Índice

Resumen	1
Descripción general	2
Objetivo general	2
Objetivos específicos	2
Situación problemática	3
Variables que monitorea el sistema	3
Entradas y salidas del sistema	3
Funcionamiento esperado	4
Estructura de datos	4
Listas	4
Diccionarios	5
Tuplas	5
Validaciones y manejo de errores	5
Captura de datos	5
Matriz NumPy	6
Procesamiento con NumPy	6
Clasificación del estado	7
Visualización y gráficos	7
Integración y simulación	7
Reporte final del programa	8
Evidencia de funcionamiento	9
Casos de prueba	9
Conclusiones	12

Resumen

El proyecto Mundial Trivia Challenge implementa un juego educativo de preguntas sobre fútbol internacional. El sistema integra una interfaz gráfica desarrollada con Tkinter, una base de 50 preguntas, cinco fases de dificultad progresiva, validación de entradas, almacenamiento estructurado de respuestas, procesamiento matricial con NumPy y visualización mediante Matplotlib. Cada partida produce un historial, una matriz numérica, indicadores de rendimiento, cuatro gráficos y un reporte final.

Además, se desarrolló un simulador reproducible para evaluar el comportamiento del programa y casos de prueba automatizados para verificar sus funciones principales.

Palabras clave: Python, NumPy, Matplotlib, Tkinter, simulación, trivia.

Descripción general

Mundial Trivia Challenge es una aplicación de escritorio desarrollada en Python que presenta preguntas sobre jugadores, selecciones, goles históricos y datos de la Copa Mundial. El jugador dispone de tres vidas y avanza por cinco fases: Fase de Grupos, Octavos de Final, Cuartos de Final, Semifinal y Final. En cada fase responde cinco preguntas y recibe una cantidad de puntos asociada con su dificultad.

El proyecto combina programación estructurada, programación orientada a eventos, estructuras de datos y análisis numérico. Python se empleó por su legibilidad y por la disponibilidad de bibliotecas científicas y gráficas ampliamente utilizadas (Python Software Foundation, [2026a](#)). NumPy permite representar el historial como una matriz y aplicar operaciones vectorizadas (Harris et al., [2020](#)), mientras que Matplotlib permite producir visualizaciones reproducibles (Hunter, [2007](#)).

Objetivo general

Desarrollar un sistema interactivo de trivia mundialista que capture, valide y procese las respuestas del jugador, calcule indicadores de rendimiento mediante NumPy y presente los resultados con reportes y gráficos generados con Matplotlib.

Objetivos específicos

- Organizar un banco de 50 preguntas mediante listas, diccionarios y tuplas.
- Implementar reglas de puntaje, vidas y avance de fases mediante variables, condicionales y ciclos.
- Validar el nombre del jugador y la estructura del banco de preguntas.
- Registrar los datos de cada respuesta en un historial estructurado.
- Convertir el historial en una matriz NumPy y calcular indicadores estadísticos.

- Clasificar el desempeño de acuerdo con el porcentaje de respuestas correctas.
- Generar gráficos y archivos de salida que faciliten la interpretación de resultados.
- Verificar las funciones principales mediante casos de prueba y simulaciones reproducibles.

Situación problemática

Las actividades de evaluación basadas únicamente en un puntaje final ofrecen poca información sobre el proceso seguido por el estudiante. No permiten identificar en qué categorías tuvo dificultades, cómo evolucionó su puntaje ni cuánto tiempo necesitó para responder. También resulta difícil comprobar manualmente el comportamiento de un programa que selecciona preguntas de forma aleatoria.

La solución propuesta no se limita a mostrar si una respuesta es correcta. El sistema conserva cada interacción y transforma los datos en estadísticas, clasificaciones y gráficos. La simulación con semilla conocida permite repetir exactamente un escenario, lo cual mejora la validación técnica.

Variables que monitorea el sistema

Entradas y salidas del sistema

La entrada manual principal es el nombre del jugador y la selección de una de cuatro alternativas. Las entradas internas incluyen el banco de preguntas, las reglas de las fases y la semilla utilizada en la simulación.

Las salidas visuales son el puntaje, las vidas, el avance del torneo, la respuesta correcta y la clasificación de rendimiento. Las salidas persistentes se almacenan en CSV, TXT y PNG: historial de respuestas, matriz NumPy, reporte final y gráficos. El simulador produce una matriz adicional con los resultados de múltiples partidas.

Cuadro 1

Variables principales monitoreadas

Variable	Tipo	Propósito
jugador	cadena	Nombre normalizado del participante.
puntaje	entero	Suma acumulada de puntos obtenidos.
vidas	entero	Oportunidades restantes antes de finalizar.
fase actual	entero	Posición actual dentro de la tupla de fases.
acierto	booleano	Resultado de la respuesta seleccionada.
tiempo de respuesta	decimal	Segundos empleados en responder.
preguntas jugadas	lista	Identificadores usados para evitar repeticiones.
historial	lista	Diccionarios con los datos de cada interacción.

Funcionamiento esperado

El programa solicita un nombre válido, crea el estado inicial y muestra la primera fase. Para cada pregunta carga una imagen cuando está disponible, presenta cuatro opciones y bloquea los botones después de responder. Un acierto suma puntos; un error resta una vida. La partida termina cuando el jugador pierde las tres vidas o completa las cinco fases. Finalmente, el sistema calcula estadísticas, clasifica el desempeño y exporta el reporte.

El flujo gráfico se implementó con Tkinter, incluido en la biblioteca estándar de Python y basado en Tcl/Tk (Python Software Foundation, 2026b).

Estructura de datos

Listas

La lista `LISTA_TRIVIA` contiene los 50 diccionarios de preguntas. También se utilizan listas para almacenar identificadores respondidos, el historial y el ranking de la sesión. La lista es apropiada porque estos elementos se recorren y aumentan durante la ejecución.

Diccionarios

Cada pregunta es un diccionario con las claves `id`, `categoria`, `dificultad`, `pregunta`, `imagen`, `opciones` y `correcta`. El estado del juego también es un diccionario. Esta estructura permite acceder a cada valor mediante una clave descriptiva.

Tuplas

Las cuatro opciones de respuesta se almacenan en una tupla porque no deben cambiar durante la partida. Las fases y los nombres de las columnas de la matriz también se representan como tuplas. La inmutabilidad expresa que estos datos corresponden a una configuración fija.

Validaciones y manejo de errores

La función `validar_nombre` elimina espacios repetidos, exige entre 2 y 40 caracteres y acepta solamente letras, espacios, guiones o apóstrofes. La función `validar_banco_preguntas` verifica que los identificadores sean únicos, que cada pregunta tenga cuatro opciones en una tupla, que la respuesta correcta sea un índice válido y que la dificultad pertenezca al conjunto permitido.

La selección de preguntas rechaza cantidades no positivas y evita solicitar más elementos de los disponibles. La carga de imágenes se encuentra protegida con `try/except`; si un archivo no existe, la interfaz utiliza un marcador visual y continúa funcionando. Las funciones estadísticas también rechazan porcentajes, categorías y dificultades inválidas.

Captura de datos

Cuando el usuario responde, la aplicación calcula el tiempo transcurrido con un reloj monótono. Después de actualizar vidas y puntaje, registra un diccionario con número de respuesta, jugador, pregunta, fase, categoría, dificultad, acierto, puntos obtenidos, puntaje acumulado, vidas restantes y tiempo.

La captura se realiza inmediatamente después de cada interacción. De esta manera, una derrota anticipada conserva todas las respuestas efectivamente contestadas y no crea registros para preguntas que nunca se mostraron.

Matriz NumPy

El historial se transforma en una matriz de tipo decimal. Las categorías y dificultades textuales se codifican mediante diccionarios para mantener una representación completamente numérica.

Cuadro 2

Columnas de la matriz de respuestas

Índice	Columna	Índice	Columna
0	Número	5	Acierto
1	Pregunta	6	Puntos obtenidos
2	Fase	7	Puntaje acumulado
3	Categoría	8	Vidas restantes
4	Dificultad	9	Tiempo de respuesta

La forma de la matriz es $n \times 10$, donde n representa la cantidad de respuestas contestadas. El uso de un arreglo homogéneo facilita seleccionar columnas, construir máscaras booleanas y aplicar agregaciones.

Procesamiento con NumPy

El módulo `analitica.py` utiliza `numpy.asarray` para crear la matriz. La suma de la columna de aciertos determina las respuestas correctas, mientras que `numpy.mean` calcula el porcentaje de efectividad. El último valor de la columna de puntaje acumulado contiene el puntaje final. La media y desviación estándar de los tiempos permiten describir la velocidad de respuesta.

Para los resultados por fase y categoría se crean máscaras booleanas. Cada máscara selecciona las filas que pertenecen a un grupo y luego se calcula la media de la columna de aciertos. Este procesamiento utiliza operaciones matriciales reales y no solamente

NumPy como contenedor.

Clasificación del estado

El sistema realiza dos clasificaciones. La primera describe el estado del torneo como `en_juego`, `derrota` o `salon_fama`. La segunda interpreta el porcentaje de aciertos:

Cuadro 3

Clasificación del desempeño

Porcentaje	Clasificación
80 % a 100 %	Excelente
60 % a 79.99 %	Bueno
40 % a 59.99 %	En proceso
0 % a 39.99 %	Necesita refuerzo

Los límites se validan mediante pruebas específicas para evitar errores en los valores de frontera.

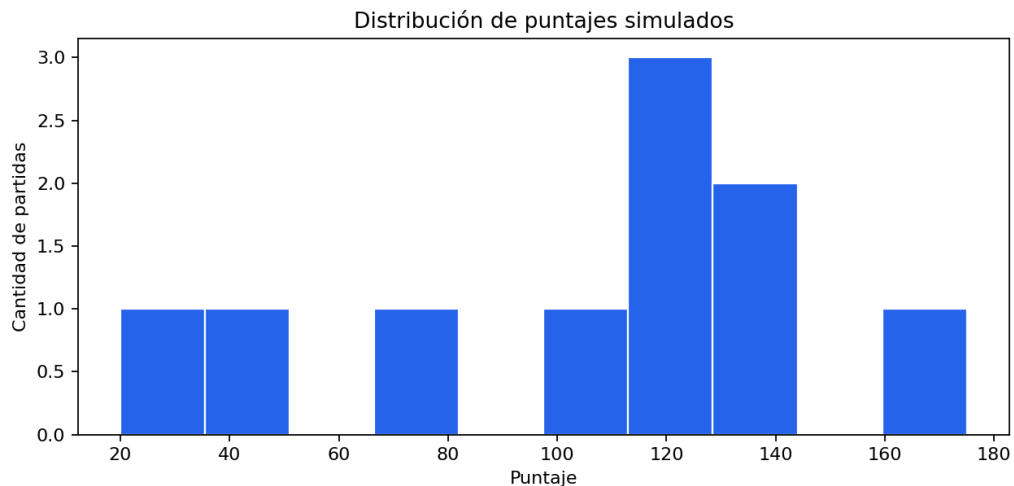
Visualización y gráficos

Matplotlib genera cuatro figuras para cada partida: cantidad de aciertos y errores, porcentaje por fase, porcentaje por categoría y evolución del puntaje. Se emplea el backend **Agg**, por lo que los gráficos pueden guardarse incluso cuando no existe una ventana gráfica. Todas las figuras se cierran después de exportarse para evitar consumo acumulado de memoria.

La simulación genera además un histograma de puntajes. La Figura siguiente corresponde a 10 partidas ejecutadas con la semilla 42.

Integración y simulación

La interfaz llama a las funciones de la lógica sin duplicar reglas. Al finalizar una partida, solicita al módulo analítico el resumen y la generación de archivos. Esta separación permite utilizar las mismas funciones desde la interfaz, las pruebas y el simulador.

**Figura 1**

Distribución de puntajes obtenidos en la simulación

El archivo `simulador.py` ejecuta partidas automáticas con probabilidades de acierto de 0.80, 0.60 y 0.40 para preguntas fáciles, medias y difíciles, respectivamente. Un objeto `random.Random` inicializado con una semilla controla tanto la selección de preguntas como las respuestas simuladas.

La ejecución de 10 partidas con semilla 42 produjo un puntaje promedio de 104.50, un máximo de 175, un mínimo de 20 y una desviación estándar de 43.27. En esa muestra no se obtuvieron victorias porque las vidas se agotaron antes de completar las cinco fases. Estos resultados no representan habilidad humana; sirven para probar caminos variables del programa de manera repetible.

Reporte final del programa

Cada partida crea la carpeta `resultados/ultima_partida` con los siguientes archivos:

- `historial_respuestas.csv`: datos descriptivos de cada respuesta.
- `matriz_numpy.csv`: representación numérica con encabezados.
- `reporte_final.txt`: resumen legible del jugador y su rendimiento.
- `graficos/*.png`: cuatro visualizaciones de la partida.

El simulador produce archivos equivalentes en `resultados/simulacion`. Los archivos CSV pueden revisarse en una hoja de cálculo y conservan un formato adecuado para análisis posteriores.

Evidencia de funcionamiento

Las siguientes capturas documentan el inicio, la presentación de preguntas, la retroalimentación y el resultado final. Las imágenes se generan mediante el mismo programa de escritorio utilizado por los jugadores.

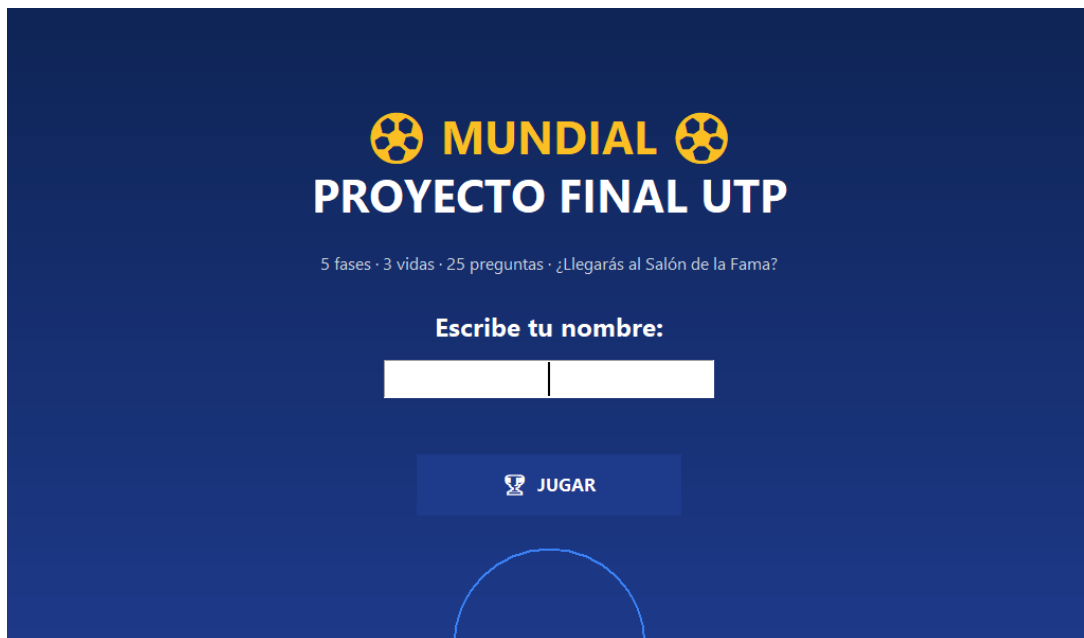


Figura 2
Pantalla de bienvenida y captura del nombre

Casos de prueba

Las pruebas fueron implementadas con `unittest`. La suite se ejecuta sin abrir la interfaz y cubre los componentes que contienen las reglas y cálculos principales. La ejecución verificada obtuvo 11 pruebas satisfactorias. También se compiló el código con `compileall` y se ejecutó la simulación desde la línea de comandos.

**Figura 3**

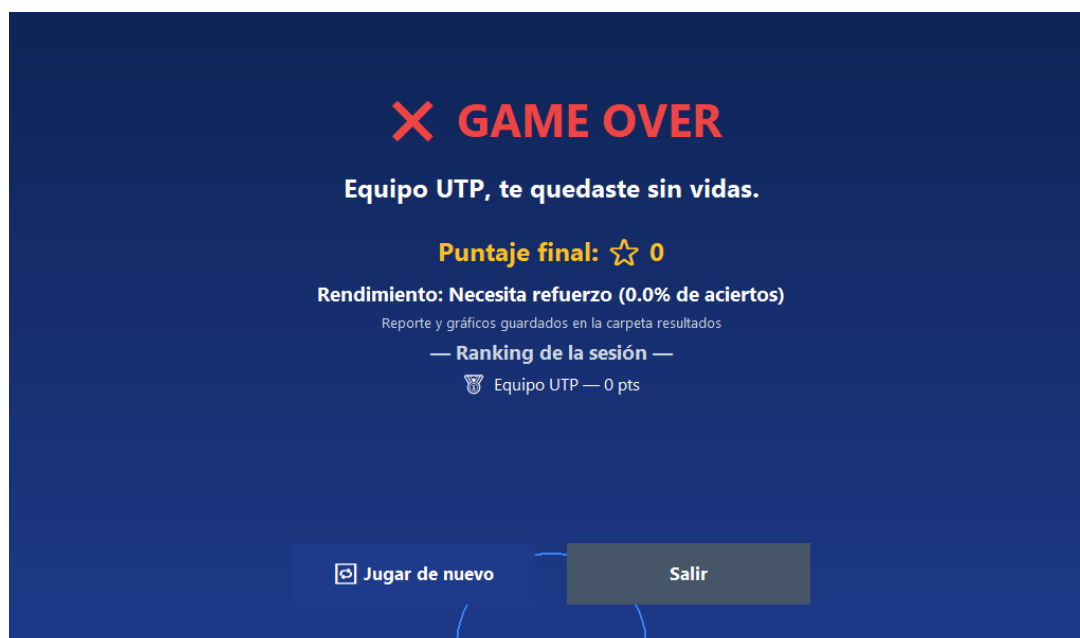
Presentación de una fase del torneo

**Figura 4**

Pregunta con imagen y cuatro alternativas

**Figura 5**

Retroalimentación visual después de responder

**Figura 6**

Pantalla final con clasificación de rendimiento

Cuadro 4*Casos de prueba automatizados*

Caso	Resultado esperado
Nombre con espacios repetidos	Normalización a un solo espacio.
Nombre compuesto por números	Excepción de validación.
Identificador de pregunta repetido	Rechazo del banco inconsistente.
Preguntas con exclusiones	Selección sin identificadores usados.
Registro de un acierto	Aumento de puntos y creación del historial.
Registro repetido	Excepción para impedir duplicidad.
Matriz con dos respuestas	Forma 2×10 e indicadores correctos.
Límites de clasificación	Categoría correcta en 40, 60 y 80 por ciento.
Exportación de partida	Creación de CSV, TXT y cuatro PNG.
Simulación con igual semilla	Secuencia de puntajes idéntica.
Cantidad de partidas igual a cero	Excepción de validación.
Exportación de simulación	Creación de cuatro artefactos.

Conclusiones

El proyecto cumple una función lúdica y, al mismo tiempo, demuestra conceptos fundamentales de programación. Las estructuras de datos tienen un propósito concreto: las listas almacenan colecciones variables, los diccionarios describen entidades y las tuplas protegen configuraciones inmutables.

La incorporación de NumPy transforma las interacciones del juego en información cuantitativa verificable. Matplotlib comunica esos resultados mediante gráficos, y la simulación permite repetir escenarios controlados. La separación entre interfaz, lógica, preguntas, análisis y simulación mejora la claridad y facilita las pruebas.

La generación documental se ejecuta dentro de contenedores Docker con versiones fijadas de TeX Live y Pandoc. Esto evita depender de instalaciones locales y hace reproducible la construcción de los archivos PDF y Word.

Referencias

- Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array Programming with NumPy. *Nature*, 585, 357-362. <https://doi.org/10.1038/s41586-020-2649-2>
- Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering*, 9(3), 90-95. <https://doi.org/10.1109/MCSE.2007.55>
- Python Software Foundation. (2026a). *Python 3.14 Documentation*. Consultado el 19 de julio de 2026, desde <https://docs.python.org/3/>
- Python Software Foundation. (2026b). *tkinter — Python Interface to Tcl/Tk*. Consultado el 19 de julio de 2026, desde <https://docs.python.org/3/library/tkinter.html>